

I. QUESTION (20 MINUTES)

```
1 public abstract class Heap {  
2     protected HeapNode array[];  
3     protected int count;  
4     protected int N;  
5 }  
6 public abstract class DHeap extends Heap {  
7     protected int d;  
8 }  
9 public class HeapNode {  
10     private int data;  
11     private int name;  
12 }
```

Given the index of a d-heap node, write a recursive Java method that returns the number of descendants (children, grandchildren, grandgrandchildren, etc.) of that heap node. Do not use any class or external methods.

```
1 int descendants(int no)
```

II. QUESTION (20 MINUTES)

```
1 class Heap {  
2 protected:  
3     HeapNode *array;  
4     int count;  
5     int N;  
6 }  
7 class HeapNode {  
8 private:  
9     int data;  
10    int name;  
11 }
```

Given the index of a heap node in a max-heap, write a recursive C++ method that determines if the subtree rooted from the node with index satisfies the heap-order property. Do not use any class or external methods.

```
1 bool heapOrder(int index)
```

III. QUESTION (20 MINUTES)

```
1 public class DisjointSet {  
2     private Set[] sets;  
3     private int count;  
4 }  
5 public class Set {  
6     private int data;  
7     private int parent;  
8     private int depth;  
9 }
```

Given the index of a set, write a Java method that returns the indexes of its grandchildren as a linked list. Do not use any class or external methods.

```
1 LinkedList grandChildren(int index)
```

IV. QUESTION (20 MINUTES)

```
1 class DisjointSet {  
2 private:  
3     Set* sets;  
4     int count;  
5 }  
6 class Set {  
7 private:  
8     int data;  
9     int parent;  
10    int depth;  
11 }
```

Given the index of a set S , write a C++ method that unmerges (creates disjoint sets of) it. After unmerging, the direct children of S and S itself will be disjoint sets themselves. You don't need to modify the depths. Do not use any class or external methods.

```
1 void unmerge(int index)
```

V. QUESTION (25 MINUTES)

1	public class Graph {	1	public class Edge {
2	private int vertexCount;	2	private int from;
3	private int [][] edges;	3	private int to;
4	}	4	private int weight;
		5	private Edge next;
		6	}
		7	public class EdgeList {
		8	private Edge head;
		9	private Edge tail ;
		10	}
		11	public class Graph {
		12	private int vertexCount;
		13	private EdgeList[] edges;
		14	}

Write a Java method to find the out-degree of a node given its index. You should write the code for both adjacency matrix and adjacency list representation. Do not use any class or external methods.

1	int outdegree(int index)
---	---

VI. QUESTION (25 MINUTES)

```

1 class Graph {
2 private:
3     int vertexCount;
4     int** edges;
5 }

```

```

1 class Edge {
2 private:
3     int from;
4     int to;
5     int weight;
6     Edge* next;
7 }
8 public class EdgeList {
9 private:
10    Edge* head;
11    Edge* tail ;
12 }
13 public class Graph {
14 private:
15     int vertexCount;
16     EdgeList* edges;
17 }

```

Write a C++ method that checks if the graph is fully connected or not. Write the function for both adjacency matrix and adjacency list representations. Do not use any class or external methods.

```

1 bool isFullyConnected()

```